

Intermediate DSA: Strings

Strings : Combination of chars X

X Collection of chars

X Groups of chars

✓ Sequence of chars

✓ Array/List of chars

X

abcd	c bad
------	-------

How we int? → binary

How we Strings → ASCII

'A' → 65	'a' → 97	'0' → 48
'B' → 66	'b' → 98	'1' → 49
⋮	⋮	⋮
'Z' → 90	'z' → 122	'9' → 57

Diagram notes: A red bracket above the lowercase letters 'a' through 'z' is labeled '32', indicating the range of lowercase ASCII values. A red 'X' is marked above the 'a' entry. A yellow bracket below the lowercase letters 'a' through 'z' is also labeled '32'.

'ab' → X String X ← '10' → 58

mutation → change

Immutable → change

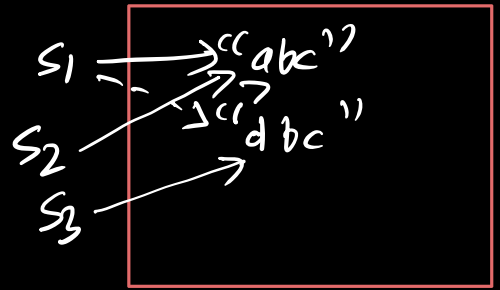
Immutable Strings

(Java / Python & other languages)

String s1 = "abc"

s1[0] = 'd'

↳ s1 = "dbc"



String pool

String s2 = "abc"

s3 = "abc"

s3[0] = 'd'

Garbage Collection

C++ Mutable String

String s1 = "abc"

s1[0] = 'd'

s1 -

d	b	c
---	---	---

Append char to immutable String

String s1 = 'a'

s1 = s1 + 'b'

s1 = s1 + 'c'

s1 = s1 + 'd'

"a"
"ab"
"abc"
"abcd"

String pool

TC: Appending 2 String of length N & M
 $O(N+M)$

Q I am creating a string of len N by adding character at a time and **Strings are immutable**
 What is TC? SC?

"abcd"

$$\Delta' = ''$$

$$\Delta' = '' + 'a' \longrightarrow 1$$

$$\Delta' = 'a' + 'b' \longrightarrow 2$$

$$\vdots \longrightarrow 3$$

$$\Delta_1 = 'abc' + 'd' \longrightarrow 4$$

$$1 + 2 + 3 + \dots + N = O(N^2)$$

$$SC: O(N^2)$$

* String Builders

String Builders $\Delta b = \text{new String Builder}();$

a b

$$\Delta b.add('a'); \longrightarrow TC = O(1)$$

$$\Delta b.add('b');$$

$$\Delta b.append('a b c d'); \longrightarrow TC = O(N)$$

$$\Delta b.toString()$$

N is len of adding string

'a b'

'b c'

'c d'

$$\Delta \Rightarrow \Delta.join()$$

$$['ab', 'bc', 'cd'].join()$$

Q :- Given a String S. Toggle the case of every character.
Upper character $\rightarrow$ Lower ch.
Lower ch. $\rightarrow$ Upper ch.

S: aBcAEd $\rightarrow$ AbCaED

toggle(S) {

for (i=0; i < S.size(); i++) {

// if S[i] is lower

if (S[i] >= 'a' && S[i] <= 'z') {

S[i] -= 32;

}

else {

S[i] += 32;

}

}

a $\rightarrow$ 97 $\Rightarrow$ A $\rightarrow$ 65
b $\rightarrow$ 98 $\Rightarrow$ B $\rightarrow$ 66

TC - $O(n)$
SC - $O(1)$

'A' → 65

'B' → 66

⋮

'Z'

0	1	0	0	0	0	0	1
7	6	5	4	3	2	1	0

↓
64

'a' → 97

'b'

⋮

'z'

1	1	0	0	0	0	1
6	5	4	3	2	1	0

↓ ↓
64 32

'z' → 122

'z' → 90

7	6	5	4	3	2	1	0
0	1	1	1	0	1	0	
0	1	0	1	1	0	1	0

↑

⇒ In every pair of small & capital char
only 5th bit will be diff

$$S[i] = S[i] \wedge (32)$$

π-d,

0100000

$$1 \wedge 3 \Rightarrow 2$$

$$1 \Rightarrow 01$$

$$3 \Rightarrow 11$$

$$\begin{array}{r} 10 \end{array} \rightarrow 2$$

$$\begin{aligned} 'a' \wedge 32 &\Rightarrow 97 \Rightarrow 1100001 \\ 32 &\Rightarrow 0100000 \\ 'A' \Leftrightarrow 65 &\Leftrightarrow 1000001 \end{aligned}$$

Q Given a string of lower case ch. Sort it in ASC Order.

S: d a b a e d b
a a b b d d e

26

$SORT(S) \Rightarrow$ Sorted $\smile$

TC: $N \log(N)$

Hint: Only 26 unique ch.

a — 1 + 1 = 2

b — 1 + 1 = 2

c — — — 0

d — 1 + 1 = 2

e — 1 = 1

a a b b d d e

Ex 2:- b c c d

a 0

b 1

c 2

d 1

b c c d

Disc/ Hashmap/ arr[26]

'a' $\rightarrow$ 0

'b' $\rightarrow$ 1

'c' $\rightarrow$ 2

⋮

'z' $\rightarrow$ 25

$\Rightarrow$

c - 'a'

$S[i] - 'a'$

'a' — 0

'z' — 25

0	1	2	3
2	1	2	1

b a c c a b d
 ↑

cnt['b']++

cnt['b' - 'a']++ $\Rightarrow$ cnt[1]++

cnt['a' - 'a']++ $\Rightarrow$ cnt[0]++

cnt['c' - 'a']++ $\Rightarrow$ cnt[2]++

cnt[0]++

a a b c c d

$\Rightarrow$ Array is better because it's always $O(1)$ access whereas in hashmap we have on average $O(1)$ and $O(N)$ in worst case.

Count Freq

int count[26] = {0}

for (i=0; i<N; i++) {

index for s[i] $\rightarrow$ s[i] - 'a'

count[s[i] - 'a']++;

}

TC: $O(N)$

Build the String as per count in disc order

$k = 0;$

for($i=0; i < 26; i++$) {

count[i] stores freq of $i + 'a'$

for($j=0; j < \text{count}[i]; j++$) {

$\Delta[k] = \text{char}(i + 'a');$
 $k++$

type casting

}

return $\Delta;$

$O(N)$

Count ans

$i=0$ a — 1
 $i=1$ b — 2
 $i=2$ c — 0
 $i=3$ d — 1

Δ

a	b	b	d
---	---	---	---

 k

$i + 'a' \Rightarrow 'a' + 'a' \Rightarrow 'b'$

$\text{count}[i] = 1$

$j \Rightarrow 0 \rightarrow 1$

TC: $O(N)$

SC: $O(26) \approx O(1)$

$O(26) \approx O(1)$

$O(26) \approx O(1)$

i	j	# times
0	[0-1a]	1 a
1	[0-1b]	1 b
...	...	...
25	[0-1z]	1 z

$1a + 1b + \dots + 1z = N$

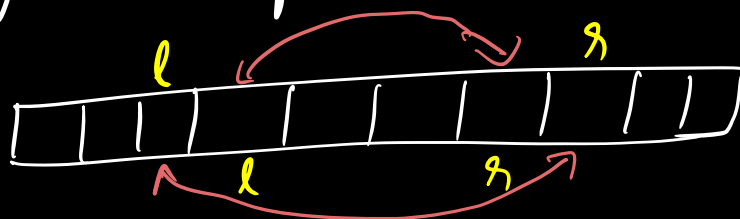
Q Given a string S & two indices l & r

Reverse the substring from l to r

0 1 2 3 4 5 6
a b d e a g f

$l: 2$ $r: 5$

Ans: a b g a e d f



↓ ↓
a b c d e f

↓ ↓
a e c d b f

↓ ↓
a e d c b f

→ termination condⁿ

```
reverse(S, l, r) {  
    while (l < r) {  
        swap(S[l], S[r]);  
        l++;  
        r--;  
    }  
}
```

TC: $O(n)$

SC: $O(1)$ ^{mutable}

$O(n)$

↓
Immutable

Q Given a character array storing a sentence.

Reverse it word by word

* No extra space is allowed

* Every word is separated by a single white space = (' ')

* No inbuilt method (reverse, split)

here _ is _ a _ bag

h | e | r | e | _ | i | s | _ | a | _ | b | a | g

Ans bag a is here

Q: "Are you as clever as I am"

Ans: "am I as clever as you Are"

=> "ma I a revelec a voy era"

=> "am I as clever as you Are"

TC : $O(N)$ $O(N)$ => $O(N)$

SC: $O(1)$

↓
Reverse
in whole
string

$w_1 \rightarrow |w_1|$

$w_2 \rightarrow |w_2|$

⋮

$(w_1 + w_2 + w_3) \approx N$

ma - I - Δa - reverse $\rightarrow$ return if you get Δp or $n \geq N$

```

    (
        call reverse( $\Delta$ , l,  $n-1$ )
         $n++$ 
         $l = n$ 
        call reverse( $\Delta$ , l,  $n-1$ )
         $n++$ 
         $l = n$ 
        call reverse( $\Delta$ , l,  $n-1$ )
         $n++$ 
         $l = n$ 
    )

```

Q:- Given a string check if a given substring is palindrome or not

S: a c d c a

$l \quad n \rightarrow \text{reverse}(S) = S$

$l = 0, n = N-1$

while ($l < n$) {

if ($S[l] \neq S[n]$)

return false;

$l++;$ $n--;$

}

a c d c a
 l
 r

a b b a
 l r

$$l = N/2, \quad r = N/2$$

while $(l > 0 \text{ and } r < N) \{$

if $(S[l] != S[r])$

return false

$l--;$

$r++;$

}

Q Given a String Calculate length of largest

Amazon
 OLA

palindromic Substring?

Ex: 0 1 2 3 4 5
 a b a c a b

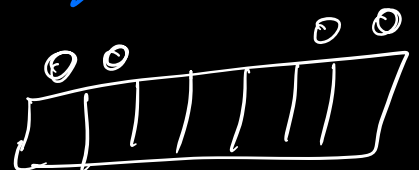
aba $\rightarrow 3$

Ans: 5

bacab $\rightarrow 5$

ab
 aba
 abac
 ca
 abb X

If a substring is palindromic
 it is called palindromic
 substring.



Ex 2: abcde

(a)

$A = \text{rev}(A)$

Can

malayalam

NITIN $\rightarrow$ NITIN POP

```
for (i = 0; i < N; i++) {
```

```
    for (j = i; j < N; j++) {
```

```
        checkP( $\Delta$ , i, j) TC =  $O(N)$ 
```

```
    }
```

```
}
```

TC = $O(N^3)$

Str:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
x	b	d	y	z	z	y	d	b	d	y	z	y	d	x
	$\uparrow$						$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$	$\uparrow$
							x				p_1	p_2		x

$\in p_1 p_2 \rightarrow$

TC = $O(N)$

```
int expand(String S, int p1, int p2) {
```

```
    while (p1 >= 0 && p2 < N && S[p1] == S[p2])
```

```
        p1--;
```

```
        p2++;
```

```
}
```

```
return p2 - p1 - 1;
```

```
}
```

0 1 2 3 4

c a b a d

p_1

p_2

$p_2 - p_1 = 4 - 1$

$$i - j \Rightarrow \boxed{j - i + 1} - 2$$

$$\Rightarrow j - 1 - 1$$

with this we can get max odd len palindromes

For even len $\rightarrow p_1 = i, p_2 = i + 1$

For odd len $\Rightarrow p_1 = i, p_2 = i$

ans = 0

For odd len

for $i = 0; i < N; i++$

ans = max(ans, expand(S, i, i)); $TC = O(N)$

}

For even len

for $i = 0; i < N - 1; i++$

ans = max(ans, expand(S, i, i + 1)); $TC = O(N)$

}

return ans

}

$TC: O(N^2)$

$SC: O(1)$

We have a better solution $O(N)$ time
Manacher's Algo

we also have a DP solution
 $O(N^2)$ time
 $O(N^2)$ Space.